Community Library Management System

Entity Models and DbContext Design (EF Core + PostgreSQL)

Folder Structure Recommendation

```
LibraryManagementSystem
│
├── Contexts
│   └── LibraryDbContext.cs
│
├── Models
│   ├── Member.cs
│   ├── Membership.cs
│   ├── MembershipPayment.cs
│   ├── Book.cs
│   ├── BookCategory.cs
│   ├── BookCopy.cs
│   ├── BookDamageHistory.cs
│   ├── Borrowing.cs
│   ├── Fine.cs
│   └── FinePayment.cs
│
├── Models/Enums
│   ├── MembershipStatus.cs
│   ├── BookCopyStatus.cs
│   ├── FineStatus.cs
│   ├── FineReason.cs
│   ├── BorrowingStatus.cs
│   └── DamageSeverity.cs
```

Enums

MembershipStatus

```
public enum MembershipStatus
{
    ACTIVE,
```

```
    EXPIRED,
    NOT_PURCHASED
}
```

BookCopyStatus

```
public enum BookCopyStatus
{
    AVAILABLE,
    BORROWED,
    DAMAGED
}
```

FineStatus

```
public enum FineStatus
{
    PENDING,
    PAID
}
```

FineReason

```
public enum FineReason
{
    LATE_RETURN,
    DAMAGED_BOOK
}
```

BorrowingStatus

```
public enum BorrowingStatus
{
    ACTIVE,
    RETURNED,
```

```
    OVERDUE
}
```

DamageSeverity

```
public enum DamageSeverity
{
    LOW,
    MEDIUM,
    HIGH
}
```

Entity Models

Member

```
public class Member
{
    public int MemberId { get; set; }

    public string Name { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public string PhoneNumber { get; set; } = string.Empty;

    public int? MembershipId { get; set; }

    public MembershipStatus MembershipStatus { get; set; }

    public DateTime? MembershipStartDate { get; set; }

    public DateTime? MembershipEndDate { get; set; }


    // Navigation Properties

    public Membership? Membership { get; set; }
```

```
    public ICollection<Borrowing> Borrowings { get; set; }
        = new List<Borrowing>();

    public ICollection<MembershipPayment> MembershipPayments { get; set; }
        = new List<MembershipPayment>();

    public ICollection<BookDamageHistory> DamageHistories { get; set; }
        = new List<BookDamageHistory>();
}
```

Membership

```
public class Membership
{
    public int MembershipId { get; set; }

    public string MembershipName { get; set; } = string.Empty;

    public decimal MembershipPrice { get; set; }

    public int BorrowLimit { get; set; }

    public int BorrowDaysLimit { get; set; }


    // Navigation Properties

    public ICollection<Member> Members { get; set; }
        = new List<Member>();
}
```

MembershipPayment

```
public class MembershipPayment
{
    public int PaymentId { get; set; }

    public int MemberId { get; set; }

    public int MembershipId { get; set; }
```

```
    public decimal PaidAmount { get; set; }

    public DateTime StartDate { get; set; }

    public DateTime EndDate { get; set; }

    public DateTime PaymentDate { get; set; }


    // Navigation Properties

    public Member Member { get; set; } = null!;

    public Membership Membership { get; set; } = null!;
}
```

BookCategory

```
public class BookCategory
{
    public int CategoryId { get; set; }

    public string CategoryName { get; set; } = string.Empty;


    // Navigation Properties

    public ICollection<Book> Books { get; set; }
        = new List<Book>();
}
```

Book

```
public class Book
{
    public int BookId { get; set; }

    public string Title { get; set; } = string.Empty;
```

```
    public string Author { get; set; } = string.Empty;

    public string ISBN { get; set; } = string.Empty;

    public int PublishedYear { get; set; }

    public int CategoryId { get; set; }


    // Navigation Properties

    public BookCategory Category { get; set; } = null!;

    public ICollection<BookCopy> BookCopies { get; set; }
        = new List<BookCopy>();
}
```

BookCopy

```
public class BookCopy
{
    public int CopyId { get; set; }

    public int BookId { get; set; }

    public string CopyNumber { get; set; } = string.Empty;

    public BookCopyStatus CopyStatus { get; set; }

    public DateTime AddedDate { get; set; }


    // Navigation Properties

    public Book Book { get; set; } = null!;

    public ICollection<Borrowing> Borrowings { get; set; }
        = new List<Borrowing>();

    public ICollection<BookDamageHistory> DamageHistories { get; set; }
        = new List<BookDamageHistory>();
}
```

BookDamageHistory

```
public class BookDamageHistory
{
    public int DamageId { get; set; }

    public int BookCopyId { get; set; }

    public int ReportedUserId { get; set; }

    public string DamageDescription { get; set; } = string.Empty;

    public DamageSeverity Severity { get; set; }

    public DateTime DamageDate { get; set; }


    // Navigation Properties

    public BookCopy BookCopy { get; set; } = null!;

    public Member ReportedUser { get; set; } = null!;
}
```

Borrowing

```
public class Borrowing
{
    public int BorrowingId { get; set; }

    public int MemberId { get; set; }

    public int CopyId { get; set; }

    public DateTime BorrowDate { get; set; }

    public DateTime DueDate { get; set; }

    public DateTime? ReturnedDate { get; set; }

    public BorrowingStatus Status { get; set; }
```

```
    // Navigation Properties

    public Member Member { get; set; } = null!;

    public BookCopy BookCopy { get; set; } = null!;

    public ICollection<Fine> Fines { get; set; }
        = new List<Fine>();
}
```

Fine

```
public class Fine
{
    public int FineId { get; set; }

    public int BorrowingId { get; set; }

    public decimal FineAmount { get; set; }

    public FineReason Reason { get; set; }

    public FineStatus PaidStatus { get; set; }

    public DateTime CreatedAt { get; set; }


    // Navigation Properties

    public Borrowing Borrowing { get; set; } = null!;

    public ICollection<FinePayment> FinePayments { get; set; }
        = new List<FinePayment>();
}
```

FinePayment

```
public class FinePayment
{
```

```
    public int PaymentId { get; set; }

    public int FineId { get; set; }

    public decimal PaidAmount { get; set; }

    public DateTime PaidDate { get; set; }


    // Navigation Properties

    public Fine Fine { get; set; } = null!;
}
```

DbContext

```
using Microsoft.EntityFrameworkCore;

public class LibraryDbContext : DbContext
{
    public DbSet<Member> Members => Set<Member>();

    public DbSet<Membership> Memberships => Set<Membership>();

    public DbSet<MembershipPayment> MembershipPayments =>
Set<MembershipPayment>();

    public DbSet<Book> Books => Set<Book>();

    public DbSet<BookCategory> BookCategories => Set<BookCategory>();

    public DbSet<BookCopy> BookCopies => Set<BookCopy>();

    public DbSet<BookDamageHistory> BookDamageHistories =>
Set<BookDamageHistory>();

    public DbSet<Borrowing> Borrowings => Set<Borrowing>();

    public DbSet<Fine> Fines => Set<Fine>();

    public DbSet<FinePayment> FinePayments => Set<FinePayment>();


    protected override void OnConfiguring(DbContextOptionsBuilder
```

```
optionsBuilder)
    {
        optionsBuilder.UseNpgsql(

"Host=localhost;Port=5432;Database=LibraryDb;Username=postgres;Password=yourpassword"
        );
    }


    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Unique Constraints

        modelBuilder.Entity<Member>()
            .HasIndex(m => m.Email)
            .IsUnique();

        modelBuilder.Entity<Member>()
            .HasIndex(m => m.PhoneNumber)
            .IsUnique();

        modelBuilder.Entity<Book>()
            .HasIndex(b => b.ISBN)
            .IsUnique();


        // Member -> Membership

        modelBuilder.Entity<Member>()
            .HasOne(m => m.Membership)
            .WithMany(ms => ms.Members)
            .HasForeignKey(m => m.MembershipId)
            .OnDelete(DeleteBehavior.SetNull);


        // Book -> Category

        modelBuilder.Entity<Book>()
            .HasOne(b => b.Category)
            .WithMany(c => c.Books)
            .HasForeignKey(b => b.CategoryId)
            .OnDelete(DeleteBehavior.Restrict);


        // BookCopy -> Book

        modelBuilder.Entity<BookCopy>()
            .HasOne(bc => bc.Book)
```

```
        .WithMany(b => b.BookCopies)
        .HasForeignKey(bc => bc.BookId)
        .OnDelete(DeleteBehavior.Cascade);


    // Borrowing -> Member

    modelBuilder.Entity<Borrowing>()
        .HasOne(br => br.Member)
        .WithMany(m => m.Borrowings)
        .HasForeignKey(br => br.MemberId)
        .OnDelete(DeleteBehavior.Restrict);


    // Borrowing -> BookCopy

    modelBuilder.Entity<Borrowing>()
        .HasOne(br => br.BookCopy)
        .WithMany(bc => bc.Borrowings)
        .HasForeignKey(br => br.CopyId)
        .OnDelete(DeleteBehavior.Restrict);


    // Fine -> Borrowing

    modelBuilder.Entity<Fine>()
        .HasOne(f => f.Borrowing)
        .WithMany(b => b.Fines)
        .HasForeignKey(f => f.BorrowingId)
        .OnDelete(DeleteBehavior.Cascade);


    // FinePayment -> Fine

    modelBuilder.Entity<FinePayment>()
        .HasOne(fp => fp.Fine)
        .WithMany(f => f.FinePayments)
        .HasForeignKey(fp => fp.FineId)
        .OnDelete(DeleteBehavior.Cascade);


    // MembershipPayment -> Member

    modelBuilder.Entity<MembershipPayment>()
        .HasOne(mp => mp.Member)
        .WithMany(m => m.MembershipPayments)
        .HasForeignKey(mp => mp.MemberId)
        .OnDelete(DeleteBehavior.Cascade);
```

```
        // MembershipPayment -> Membership

        modelBuilder.Entity<MembershipPayment>()
            .HasOne(mp => mp.Membership)
            .WithMany()
            .HasForeignKey(mp => mp.MembershipId)
            .OnDelete(DeleteBehavior.Restrict);


        // DamageHistory -> BookCopy

        modelBuilder.Entity<BookDamageHistory>()
            .HasOne(dh => dh.BookCopy)
            .WithMany(bc => bc.DamageHistories)
            .HasForeignKey(dh => dh.BookCopyId)
            .OnDelete(DeleteBehavior.Cascade);


        // DamageHistory -> Member

        modelBuilder.Entity<BookDamageHistory>()
            .HasOne(dh => dh.ReportedUser)
            .WithMany(m => m.DamageHistories)
            .HasForeignKey(dh => dh.ReportedUserId)
            .OnDelete(DeleteBehavior.Restrict);
    }
}
```

PostgreSQL DateTime Best Practices

Recommended Approach

Use:

```
DateTime.UtcNow
```

instead of:

```
DateTime.Now
```

This avoids timezone inconsistencies.

Recommended Usage

```
borrowing.BorrowDate = DateTime.UtcNow;
```

```
fine.CreatedAt = DateTime.UtcNow;
```

```
payment.PaymentDate = DateTime.UtcNow;
```

PostgreSQL Timestamp Configuration

Recommended Fluent Configuration:

```
modelBuilder.Entity<Borrowing>()
    .Property(b => b.BorrowDate)
    .HasColumnType("timestamp");
```

Apply similarly for:

- DueDate
- ReturnedDate
- PaymentDate
- DamageDate
- MembershipStartDate
- MembershipEndDate

Nullable DateTime Usage

Use nullable DateTime for optional values.

Example:

```
public DateTime? ReturnedDate { get; set; }
```

This is useful because books may not yet be returned.

Important Note

No special pgAdmin configuration is required for DateTime handling.

EF Core + Npgsql automatically maps DateTime values correctly to PostgreSQL timestamp columns.

Best Practices Followed

1. Navigation Properties Added

- Enables lazy/eager loading
- Proper EF Core relationship tracking

---

2. Enums Used Instead of Strings

- Type safety
- Cleaner business logic
- Avoid typo bugs

---

3. Unique Constraints Added

- Email
- Phone Number
- ISBN

Prevents duplicate data.

---

4. Nullable Membership

```
public int? MembershipId
```

Allows member registration before purchasing membership.

---

5. Proper Delete Behaviors

Restrict

Used where accidental deletion is dangerous.

Cascade

Used where child data should automatically delete.

SetNull

Used for optional relationships.

---

6. Collections Initialized

```
= new List<Entity>()
```

Avoids null reference issues.

---

7. Decimal Used for Money

```
decimal
```

Never use float/double for currency.

---

8. DateTime? Used for Nullable Dates

```
DateTime?
```

Useful for returned books and optional membership dates.

Recommended Next Steps

1. Create Repository Interfaces
2. Create Repository Implementations
3. Create Service Layer

4. Implement Borrow Transaction Logic
5. Add PostgreSQL Function
6. Add Exception Handling
7. Create Console Menus
8. Add Reports